PORTAL PEC

Portal de Pasantías

Universidad Central de Venezuela

Documentación Técnica del Sistema

Introducción

El Portal PEC (Pasantías de la Escuela de Computación) es una aplicación web desarrollada con Django para gestionar el proceso de pasantías de la Universidad Central de Venezuela (UCV). La plataforma conecta a tres tipos de actores: estudiantes (pasantes), empresas/dependencias que ofrecen vacantes y administradores del sistema.

El sistema permite publicar y buscar ofertas de pasantía, postular candidatos, guardar vacantes de interés, gestionar el estado de cada aplicación y generar reportes administrativos.

1.1 Objetivos del Sistema

- Centralizar la gestión de ofertas de pasantías en un portal único de la UCV.
- Permitir a estudiantes registrarse, completar su perfil y postularse a vacantes.
- Permitir a empresas y dependencias universitarias publicar y administrar sus ofertas.
- Proporcionar al administrador visibilidad total del proceso y herramientas de gestión.
- Generar reportes en PDF con estadísticas del ciclo de pasantías por semestre.

1.2 Tecnologías Utilizadas

Componente	**Tecnología**
Framework backend	Django 4.x (Python)
Base de datos	SQLite (desarrollo)
Almacenamiento de medios	De forma local.
Archivos estáticos	Whitenoise + CompressedStaticFilesStorage
Etiquetado (tags)	django-taggit
Generación de PDFs	ReportLab
Análitica de visitas	django-user-visit
Configuración de entorno	django-environ (.env)

2. Arquitectura del Proyecto

El proyecto sigue la arquitectura MTV (Model–Template–View) propia de Django, con una estructura de aplicaciones independientes dentro del directorio apps/. La configuración está separada por entornos (base, local, test, producción).

2.1 Estructura de Directorios

Ruta	Contenido
apps/account/	Módulo de autenticación y perfiles de usuario
apps/jobapp/	Módulo principal de ofertas y postulaciones
apps/core/	Modelos abstractos reutilizables (timestamps, soft delete)
config/	Settings, URLs raíz, WSGI/ASGI
template/	Plantillas HTML globales y por módulo
static/	CSS, JS, imágenes y documentos estáticos
media/	Archivos subidos por usuarios (logos, CVs)
.env / .env.example	Variables de entorno
requirements.txt	Dependencias Python del proyecto

2.2 Configuración por Entornos

La carpeta config/settings/ contiene cuatro archivos de configuración:

- base.py → Configuración compartida por todos los entornos: apps instaladas, middleware, templates, base de datos por defecto, validadores de contraseñas, CKEditor 5 y demás parámetros comunes.
- local.py → Activa DEBUG=True, incluye django-debug-toolbar y sirve archivos de media localmente.
- test.py → Configuración orientada a las pruebas automatizadas.

La selección del entorno se realiza mediante la variable de entorno DJANGO_SETTINGS_MODULE.

2.3 Configuración de Rutas (URLs)

El enrutamiento principal en config/urls.py incluye las rutas de las dos aplicaciones, el panel de administración de Django, las URLs de CKEditor 5, el sitemap XML y el archivo robots.txt. En modo DEBUG se agregan las URLs del toolbar.

3. Módulo Core

El módulo core (apps/core/) no expone vistas ni URLs propias. Su propósito es proveer modelos abstractos que son heredados por los demás módulos para garantizar consistencia en el comportamiento de auditoría y borrado.

3.1 Modelos Abstractos

TimeStampedModel

Añade automáticamente dos campos de fecha a cualquier modelo que lo herede:

- created_at — Fecha y hora de creación del registro (auto_now_add=True).
- updated_at — Fecha y hora de última modificación (auto_now=True).

SoftDeleteModel

Implementa el patrón de borrado lógico (soft delete), que marca los registros como eliminados sin borrarlos físicamente de la base de datos. Expone dos campos y dos métodos:

- is_deleted (BooleanField) — Indica si el registro está borrado lógicamente.
- deleted_at (DateTimeField) — Marca temporal del momento del borrado.
- delete() — Sobrescribe el borrado estándar de Django: en lugar de eliminar la fila, establece is_deleted=True y registra deleted_at con la hora actual.
- hard_delete() — Ejecuta el borrado físico real cuando es estrictamente necesario.

Ambos modelos son abstractos (abstract = True), por lo que Django no crea tablas para ellos; únicamente contribuyen sus campos y comportamientos a los modelos concretos que los heredan.

4. Módulo Account

Este módulo gestiona toda la lógica de identidad del sistema: registro, autenticación, sesiones y perfiles. Reside en apps/account/ y expone sus rutas bajo el namespace account.

4.1 Modelo de Usuario (User)

Se extiende AbstractUser de Django eliminando el campo username y usando el email como identificador único (USERNAME_FIELD = "email"). Los campos adicionales son:

Campo	Tipo	Descripción
email	EmailField (unique)	Identificador principal. Reemplaza username.
role	CharField	Rol del usuario: 'employee', 'employer' o 'admin'.
gender	CharField	Género: 'M' (Male) o 'F' (Female).
tipo_empresa	CharField (nullable)	Rol que diferencia si el ente que ofrece la vacante es una 'dependencia' o 'empresa_externa'.

CustomUserManager

Reemplaza el UserManager por defecto de Django para operar sin username. Provee create_user() y create_superuser(), asignando automáticamente el rol admin y el género M al superusuario.

4.2 Perfiles de Usuario

Cada usuario tiene un perfil complementario que se crea automáticamente vía señales Django (post_save).

EmployeeProfile (Perfil de Pasante)

- cedula → Número de cédula venezolana de 8 dígitos (validado con RegexValidator).
- resume → Archivo CV subido al directorio resumes/.
- bio → Texto libre de presentación del pasante.
- skills → Etiquetas de habilidades gestionadas por django-taggit (permite búsqueda por etiqueta).

EmployerProfile (Perfil de Empresa/Dependencia)

- company_website →URL del sitio web corporativo.
- company_logo → Imagen del logotipo subida a logos/.

- description →Descripción libre de la organización.
- privilegios (BooleanField) →Indicador de empresa patrocinadora. Cuando es True, todas sus ofertas son marcadas con priority=True y aparecen destacadas en la plataforma.
- sector_empresa →Sector económico de la empresa (industria, tecnología, salud, finanzas, etc.).

4.3 Señales (Signals)

El archivo signals.py conecta dos receptores al evento post_save del modelo User. El primero (create_user_profile) crea automáticamente el perfil correspondiente la primera vez que se guarda un usuario según su rol. El segundo (save_user_profile) realiza el guardado al perfil cuando el usuario es actualizado.

4.4 Formularios de Autenticación

EmployeeRegistrationForm

Utiliza UserCreationForm e incluye el campo adicional de cédula. En el método save() asigna automáticamente role="employee" y crea o actualiza el EmployeeProfile asociado con la cédula ingresada.

EmployerRegistrationForm

Formulario de registro para empresas y dependencias. Incluye el campo tipo_empresa para distinguir entre una empresa externa y una dependencia universitaria. Al guardar, asigna role="employer".

UserLoginForm

Hereda de AuthenticationForm de Django. Customiza los mensajes de error para presentarlos en español y ajusta los placeholders del formulario.

4.5 Validación de Contraseña

SpanishMinimumLengthValidator extiende MinimumLengthValidator de Django para traducir el mensaje de error "contraseña demasiado corta" al español. Mínimo exigido: 8 caracteres.

4.6 Vistas de Account

Vista	URL	Función
UserLoginView	/login/	Login con redirección al dashboard o a la URL next.
UserLogoutView	/logout/	Cierra la sesión y redirige al login.
EmployeeRegistrationView	/employee/register/	Registro de estudiantes/pasantes.

EmployerRegistrationView	/employer/register/	Registro de empresas y dependencias.
EmployeeEditProfileView	/profile/edit/<id>/	Edición de perfil del pasante (protegida por rol).
EmployerEditProfileView	/employer/profile/edit/<id>/	Edición del perfil de empresa (protegida por rol).
CandidateProfileView	/candidate/profile/<id>/	Vista pública del perfil del pasante.
EmployerProfileView	/employer/profile/view/<id>/	Vista pública del perfil de la empresa.

5. Módulo JobApp

Es el módulo principal del sistema. Concentra la lógica de ofertas de pasantía, postulaciones, marcadores y el dashboard. Reside en apps/jobapp/ y expone sus rutas bajo el namespace jobapp.

5.1 Modelos

Category

Modelo simple con un único campo name. Las categorías se usan para clasificar las ofertas de pasantía y sirven como filtro en la búsqueda. Se gestionan íntegramente desde el panel de administración de Django.

Job (Oferta de Pasantía)

Modelo central del sistema. Hereda de TimeStampedModel y SoftDeleteModel. Sus campos principales son:

Campo	Tipo	Descripción
user	ForeignKey (User)	Empresa/Dependencia que publicó la oferta.
title	CharField	Título de la oferta (indexado).
description	CKEditor5Field	Descripción completa con formato enriquecido.
tags	TaggableManager	Etiquetas para sugerencias de pasantías relacionadas.
location	CharField	Ubicación geográfica de la oferta (indexado).
job_type	CharField (choices)	Tipo: 'Tiempo Completo', 'Medio Tiempo' o 'Flexible'.
work_mode	CharField (choices)	Modalidad: 'Remoto', 'Híbrido' o 'Presencial'.
category	ForeignKey (Category)	Categoría/área de computación de la oferta.
salary	BooleanField	Indica si la pasantía ofrece remuneración.
tipo_empresa	CharField (nullable)	Tipo de organización publicante ('dependencia' / 'empresa_externa').

priority	BooleanField	Si es True, la oferta aparece destacada (empresa patrocinadora).
views_count	PositiveIntegerField	Contador de vistas. Incrementa con cada visita al detalle.
last_date	DateField	Fecha límite de postulación.
is_published	BooleanField	La oferta es visible públicamente solo si este campo es True.
is_closed	BooleanField	La oferta fue cerrada/completada por la empresa/dependencia.

El método save() de Job invalida el caché del objeto cuando este es actualizado. El método delete() realiza un borrado lógico en cascada, marcando también como eliminadas todas las postulaciones y marcadores relacionados.

La clase Meta define índices compuestos en (title, location) y (is_published, is_closed) para optimizar las consultas más frecuentes.

Applicant (Postulación)

Registra que un usuario pasante se postuló a una oferta de pasantía. Hereda de TimeStampedModel y SoftDeleteModel. La combinación (user, job) es única. El campo status sigue el ciclo: Pendiente→ Aceptado / Rechazado.

BookmarkJob (Oferta Guardada)

Permite a los pasantes guardar ofertas de interés para revisarlas más tarde. Igual que Applicant, implementa soft delete y unicidad por (user, job). La re-adición de un marcador borrado lógicamente reactiva el registro original.

5.2 Capa de Permisos

El archivo permission.py define los mecanismos de control de acceso por rol:

- **EmployerRequiredMixin** → Mixin para vistas basadas en clase (CBV). Verifica que el usuario esté autenticado y que su rol sea de pasante. Si no se cumple, retorna HTTP 403.
- **EmployeeRequiredMixin** → Mixin para vistas basadas en clase (CBV). Verifica que el usuario esté autenticado y que su rol sea de Empresa/Dependencia. Si no se cumple, retorna HTTP 403.
- **user_is_employer / user_is_employee** → Funciones de validación equivalentes para vistas basadas en función (FBV). Elevan PermissionDenied si el rol no coincide.

5.3 Capa de Selectors y Services

El proyecto separa la lógica de consulta (selectors.py) y la lógica de mutación (services.py) de las vistas, siguiendo un patrón de arquitectura limpia ligero.

Selectors (consultas)

- get_listed_jobs() → Devuelve todas las ofertas activas (no eliminadas, no cerradas), con select_related a category y user, ordenadas por última actualización.
- search_jobs(title_or_company, company_type, category_id) — Búsqueda filtrada por título/empresa, tipo de organización y/o categoría. Combina filtros con encadenamiento de querysets (OR entre title y company_name).

Services (mutaciones)

- toggle_job_status(user_id, job_id) — Marca una oferta como cerrada (is_closed=True). Verifica que la oferta pertenezca al usuario que realiza la acción.

5.4 Template Tags Personalizados

El módulo registra cuatro tags de plantilla para su uso en los templates HTML:

- is_job_already_applied (tag simple) — Devuelve True si el usuario ya tiene una postulación activa para esa oferta.
- is_job_already_saved (tag simple) — Devuelve True si el usuario ya tiene un marcador activo para esa oferta.
- get_bookmark_id (tag simple) — Retorna el ID del BookmarkJob del usuario para esa oferta (necesario para el enlace de eliminación).
- get_application_id (tag simple) — Retorna el ID del Applicant del usuario para esa oferta (necesario para retirar la postulación).

5.5 Vistas del Módulo Jobapp

Vistas Públicas (sin autenticación requerida)

Vista	URL	Función
home_view	/	Página principal. Muestra ofertas destacadas (priority) y regulares.
JobListView	/jobs/	Listado completo de ofertas activas. Soporta filtro por user_id.

SingleJobView	/job/<id>/	Detalle de una oferta. Incrementa views_count y carga jobs relacionados por etiquetas.
SearchResultView	/result/	Resultados de búsqueda por título, empresa, tipo y categoría.
about_view	/about/	Página de información y documentos estáticos del portal.

Vistas de la Empresa / Dependencia (EmployerRequiredMixin)

Vista	URL	Función
CreateJobView	/job/create/	Crea una nueva oferta. Asigna tipo_empresa y priority del Empresa/Dependencia.
JobEditView	/dashboard/employer/job/edit/<id>	Edita una oferta propia existente.
DeleteJobView	/dashboard/employer/delete/<id>/	Realiza soft delete de la oferta e invalida su caché.
MakeCompleteJobView	/dashboard/employer/close/<id>/	Marca la oferta como cerrada (concluida).
AllApplicantsView	/dashboard/employer/job/<id>/applicants/	Lista todos los postulantes a una oferta.
ApplicantDetailsView	/dashboard/employer/applicant/<id>/	Detalle del perfil de un postulante.
UpdateApplicantStatusView	/dashboard/employer/applicant/update-status/<id>/	Cambia el estado de una postulación a 'accepted' o 'rejected'.

Vistas del Pasante (EmployeeRequiredMixin)

Vista	URL	Función
ApplyJobView	/apply-job/<id>/	Postula al pasante. Gestiona duplicados y restaura postulaciones borradas por el pasante.
JobBookmarkView	/bookmark-job/<id>/	Guarda una oferta como favorita. Manejo de duplicados igual que Apply.
DeleteApplicantView	/dashboard/employee/delete-application/<id>/	Retira (soft delete) una postulación propia.
DeleteBookmarkView	/dashboard/employee/delete-bookmark/<id>/	Elimina (soft delete) un marcador propio.

Vistas Compartidas (login requerido)

Vista	URL	Función
dashboard_view	/dashboard/	Panel unificado que adapta su contenido al rol del usuario.
admin_report_view	/dashboard/admin/report/	Genera y descarga un PDF con estadísticas y detalle de aplicaciones.
toggle_employer_privileges	/dashboard/admin/employer/<id>/privileges/<action>/	Otorga o revoca privilegios de patrocinador ('grant'/'revoke').
delete_user_view	/dashboard/admin/user/<id>/delete/	Elimina permanentemente una cuenta de usuario (solo admin).

6. Dashboard y Roles de Usuario

La vista dashboard_view actúa como punto central de control para los tres tipos de usuario. La misma URL (/dashboard/) renderiza contenido diferente según el rol:

6.1 Dashboard del Pasante

- Lista de ofertas a las que se ha postulado (appliedjobs), con su estado actual (Pendiente / Aceptado / Rechazado).
- Lista de ofertas guardadas como favoritas (savedjobs).
- Opciones para retirar postulaciones o eliminar marcadores.

6.2 Dashboard para la Empresa/Dependencia

- Lista de todas las ofertas publicadas por la Empresa/Dependencia con sus contadores de postulantes.
- Acciones rápidas para editar, cerrar o eliminar cada oferta.
- Acceso a la lista de postulantes y al cambio de estado de cada aplicación.

6.3 Dashboard del Administrador

El administrador ve un panel con estadísticas globales del sistema:

- Total de empresas externas registradas.
- Total de dependencias universitarias registradas.
- Total de estudiantes (pasantes) registrados.
- Total de administradores del sistema.
- Total de ofertas publicadas.
- Total de postulaciones activas.

Además, el panel muestra los 5 usuarios más recientemente registrados y provee controles para otorgar/revocar privilegios de patrocinador y eliminar cuentas.

6.4 Sistema de Privilegios de la Empresa/Dependencia

Cuando el administrador otorga privilegios a una Empresa/Dependencia mediante toggle_employer_privileges con acción "grant":

- El campo de privilegios del EmployerProfile se establece en True.
- Todas las ofertas activas de la Empresa/Dependenciason actualizadas a priority=True en una sola operación de BD.
- Las nuevas ofertas creadas por esa Empresa/Dependencia heredarán priority=True automáticamente en CreateJobView y JobEditView.

Al revocar (acción "revoke"), el proceso se invierte: privilegios=False y todas las ofertas vuelven a priority=False.

7. Página Principal y Motor de Búsqueda

7.1 Página Principal (home_view)

La vista home_view obtiene las ofertas activas y las divide en dos grupos: promovidas (priority=True) y regulares (priority=False). Soporta paginación AJAX para las ofertas regulares: si la solicitud incluye el encabezado X-Requested-With: XMLHttpRequest, la vista devuelve los datos de la página solicitada en formato JSON en lugar de HTML completo.

7.2 Detalle de Oferta y Caché (SingleJobView)

Cada visita a una oferta incrementa views_count directamente en la BD con un UPDATE atómico. El objeto Job completo se mantiene en caché con su id como clave por 15 minutos.

La sección de "Ofertas relacionadas" usa el sistema de etiquetas similares de django-taggit (tags.similar_objects()) para sugerir otras ofertas que comparten etiquetas.

7.3 Búsqueda de Ofertas (SearchResultView)

La búsqueda recibe tres parámetros GET opcionales:

- job_title_or_company_name → Filtra por título de oferta (icontains) o nombre de empresa (icontains), combinando ambas condiciones con OR.
- tipo_empresa — Filtra por tipo de organización (dependencia / empresa_externa).
- category — Filtra por ID de categoría.

Los filtros son acumulativos: si se proporcionan varios, todos se aplican en secuencia.

8. Generación de Reportes en PDF

La vista admin_report_view (accesible solo para el rol admin) genera y descarga un reporte PDF usando la librería ReportLab. El documento contiene las siguientes secciones:

8.1 Contenido del Reporte

- Encabezado con el email del administrador que genera el reporte y la fecha/hora.
- Estadísticas globales: total de empresas, dependencias, estudiantes, administradores, ofertas y postulaciones.
- Conteo de estudiantes con al menos una postulación aceptada vs. estudiantes sin postulaciones aceptadas.
- Listados por semestre del año en curso: empresas externas, dependencias y estudiantes que se registraron en el primer (enero–junio) o segundo semestre (julio–diciembre).
- Detalle de aplicaciones: para cada postulación activa se registra el nombre del estudiante, la oferta, la empresa/dependencia y el estado (Aceptado/Rechazado).

8.2 Paginación del PDF

El reporte gestiona la paginación manualmente: antes de escribir cada línea, verifica si la posición vertical (y) está por debajo del margen inferior. Si es así, llama a pdf.showPage() para crear una nueva página y reinicia la posición. Las líneas largas son divididas en segmentos de 90 caracteres.

9. SEO, Sitemap y robots.txt

El portal incluye varias características orientadas al posicionamiento en motores de búsqueda:

- sitemap.xml — Implementado con django.contrib.sitemaps. La clase JobSitemap (apps/jobapp/sitemaps.py) genera entradas para todas las ofertas publicadas y activas, facilitando su indexación.
- robots.txt — Generado dinámicamente desde una vista Python. Bloquea el acceso al panel de administración (/admin/) e incluye la URL del sitemap de forma automática según el esquema (http/https) y el host.
- django.contrib.sites — Integrado para soporte de URLs absolutas en el sitemap.

10. Panel de Administración de Django

El panel /admin/ está personalizado para las necesidades del Portal PEC:

10.1 Gestión de Usuarios (UserAdmin)

- Reemplaza el formulario estándar de Django por AddUserForm y UpdateUserForm adaptados para el modelo User sin username.
- La columna role_display muestra etiquetas legibles en español: "Pasante", "Administrador", "Dependencia" o "Empresa".
- Permite buscar por email, nombre y apellido.

10.2 Gestión de Ofertas (JobAdmin)

- Muestra en listado: título, prioridad, estado publicado/cerrado y fechas de creación/actualización.
- El formulario de edición incluye la sección "Soft delete" para inspeccionar o restaurar registros borrados lógicamente.

10.3 Gestión de Postulaciones (ApplicantAdmin)

- Lista con columnas: oferta, usuario, estado y si está eliminada lógicamente.
- Filtros laterales por estado (pending/accepted/rejected) e is_deleted.

10.4 Otras Entidades

- Category — Registro simple de categorías sin configuración adicional.
- BookmarkJob (BookmarkJobAdmin) — Lista con oferta, usuario y fecha de creación.

11. Variables de Entorno

La configuración sensible del proyecto se gestiona mediante un archivo .env en la raíz del proyecto. El archivo .env.example documenta todas las variables esperadas:

Variable	**Valor por defecto**	**Descripción**
SECRET_KEY	(inseguro dev)	Clave secreta de Django.
DEBUG	True	Activa modo depuración.
ALLOWED_HOSTS	127.0.0.1, localhost	Hosts permitidos (lista CSV).
DATABASE_URL	sqlite:///db.sqlite3	URL de conexión a la base de datos.
SECURE_SSL_REDIRECT	False	Redirige HTTP a HTTPS en producción.
SESSION_COOKIE_SECURE	False	Cookie de sesión solo por HTTPS.
CSRF_COOKIE_SECURE	False	Cookie CSRF solo por HTTPS.

12. Flujos de Negocio Principales

12.1 Registro y Onboarding

- El pasante accede a /employee/register/, completa nombre, apellido, correo, contraseña, género y cédula. Al guardar, se crea el User con role="employee" y se crea automáticamente su EmployeeProfile.
- La Empresa/Dependencia accede a /employer/register/, proporciona los datos de la empresa y su tipo (empresa externa o dependencia). Al guardar, se crea el User con role="employer" y su EmployerProfile.
- Ambos tipos de usuario son redirigidos al login tras el registro exitoso.

12.2 Publicación y Gestión de Ofertas

- La Empresa/Dependencia autenticado accede a /job/create/, completa el formulario con título, descripción (CKEditor), etiquetas, categoría, modalidad, fecha límite y URL de la empresa.
- Al guardar, tipo_empresa y priority se asignan automáticamente desde el perfil de la Empresa/Dependencia. La oferta queda en estado is_published=False hasta que el administrador la active desde el panel /admin/.
- La Empresa/Dependencia puede editar, cerrar o eliminar sus ofertas desde el dashboard.

12.3 Postulación de Pasantes

- El pasante autenticado accede a la página de detalle de una oferta (/job/<id>/).
- Al presionar "Postularme", se envía un POST a /apply-job/<id>/. La vista verifica que no exista ya una postulación activa. Si hay una postulación borrada lógicamente, la restaura en lugar de crear un duplicado.
- El estado inicial de la postulación es "pending". La Empresa/Dependencia puede cambiar el estado a "accepted" o "rejected" desde su dashboard.

12.4 Sistema de Marcadores

- El pasante puede guardar una oferta con el botón de marcador, que envía un POST a /bookmark-job/<id>/. La lógica de duplicados y restauración es idéntica a la de postulaciones.
- Los marcadores guardados aparecen en el dashboard del pasante y pueden eliminarse desde allí.

12.5 Gestión Administrativa

- El administrador puede otorgar o revocar privilegios a una Empresa/Dependencia desde su dashboard. Esto actualiza en masa el campo priority de todas sus ofertas.
- El administrador puede eliminar cuentas de usuario (excepto la propia) desde /dashboard/admin/user/<id>/delete/.
- El administrador puede descargar el reporte PDF completo desde /dashboard/admin/report/.

13. Recomendaciones y Mejoras Futuras

Esta sección propone mejoras para futuras versiones del Portal PEC. Las recomendaciones parten del estado actual del sistema y buscan fortalecer el control del proceso de pasantías, la calidad de los datos y la administración de la plataforma.

13.1 Enlace a LinkedIn en el Perfil del Estudiante

Situación actual. El perfil del estudiante no incluye un enlace a redes profesionales.

Mejora propuesta. Agregar un campo opcional para el enlace a LinkedIn en el perfil de los estudiantes.

- Permitiría a las empresas y dependencias conocer mejor al candidato al evaluar su postulación.
- Se debe validar que el enlace corresponda realmente a LinkedIn.
- El enlace debe mostrarse en el perfil público del estudiante, en el detalle de cada postulante y en el formulario de edición de perfil.

13.2 Reporte Administrativo por Semestre con Tutores y Empresas

Situación actual. El reporte en PDF muestra estadísticas globales y listados de registros por semestre, pero no presenta una lista consolidada de los estudiantes inscritos en el programa. Tampoco indica qué tutor tiene asignado cada estudiante ni qué empresa o dependencia eligió. Además, el sistema no tiene un registro de los tutores de la escuela; esta relación se maneja fuera de la plataforma.

Mejora propuesta. Rediseñar el reporte para incluir una lista organizada de todos los estudiantes inscritos, agrupada por semestre. Cada registro debería mostrar:

- Nombre completo y cédula del estudiante.
- Empresa o dependencia seleccionada para la pasantía.
- Tutor asignado.
- Oferta de pasantía correspondiente.
- Estado de la pasantía.

Registro de tutores.

- El administrador puede registrar, editar y desactivar a los tutores disponibles en la escuela desde el panel del portal.
- Cada tutor debería tener, como mínimo, nombre, correo de contacto y estado de disponibilidad.
- Se puede definir un cupo máximo de estudiantes por tutor para cada semestre.
- Opcionalmente, se puede indicar el área de especialidad de cada tutor (por ejemplo, desarrollo de software, redes o bases de datos).

13.3 Restricción del Registro de Estudiantes al Semestre Vigente

Situación actual. Cualquier persona puede registrarse como estudiante. Solo se valida el formato de la cédula, sin verificar que el estudiante esté inscrito en el programa de pasantías del semestre en curso.

Mejora propuesta. Limitar el registro a los estudiantes habilitados para cursar la pasantía en el semestre actual. Se proponen dos enfoques, que pueden combinarse:

- **Lista de habilitados.** El administrador carga por semestre la lista de estudiantes autorizados (por ejemplo, mediante un archivo con cédula y correo). El registro solo se completa si los datos del estudiante coinciden con dicha lista.
- **Periodo de inscripción.** Definir fechas de apertura y cierre del registro para cada semestre, gestionadas por el administrador. Fuera de ese periodo, el registro no estaría disponible.

Consideraciones adicionales.

- Definir qué ocurre con las cuentas de semestres anteriores: desactivarlas, archivarlas o conservarlas en modo de solo lectura.

13.4 Registro de Empresas por Invitación

Situación actual. Cualquier empresa o dependencia puede crear una cuenta. Aunque las ofertas requieren aprobación del administrador antes de publicarse, las cuentas se crean sin ningún filtro previo.

Mejora propuesta. Restringir el registro a las organizaciones previamente seleccionadas por la coordinación del programa, mediante un enlace de invitación.

- El administrador genera un enlace de registro único para cada organización.
- El enlace debe tener fecha de vencimiento y poder usarse una sola vez.
- Sin un enlace válido, el formulario de registro de empresas no debe estar accesible.
- Opcionalmente, la invitación puede definir de antemano el correo, el nombre de la organización y su tipo (empresa externa o dependencia), para evitar inconsistencias.
- El administrador debe poder consultar el estado de cada invitación (enviada, usada o vencida) y revocarla si es necesario.
- El envío del enlace puede automatizarse por correo electrónico.

13.5 Interfaz para el Registro de Nuevos Administradores

Situación actual. Actualmente, las cuentas de administrador solo pueden crearse desde la consola del servidor o desde el panel de administración de Django. El portal no cuenta con una interfaz propia para este fin.

Mejora propuesta. Incorporar en el panel del administrador una sección para crear nuevas cuentas de administrador.

- Debe estar disponible únicamente para usuarios con rol de administrador.

- La cuenta puede crearse definiendo una contraseña temporal, o mediante una invitación por correo para que el nuevo administrador establezca su propia contraseña.
- Se recomienda llevar un registro de auditoría que indique quién creó cada cuenta y cuándo.
- Debe ser posible desactivar o revocar el rol de administrador, manteniendo la regla actual que impide a un administrador eliminar su propia cuenta.
- Se recomienda exigir una política de contraseñas más estricta para este rol.